Quantum Algorithms for Patent Portfolio Analysis Using OpenModelica and IBM Qiskit

Mark R. Myers, J.D., Ph.D.

McCormick, Paulding & Huber PLLC
Hartford, Connecticut, U.S.A.
myers@ip-lawyers.com

Zenodo:10.5281/zenodo.22034029 v1 posted 7 Sept. 2026

Abstract

Near-term quantum computers are hybrid machines: classical computers, microprocessors, and control hardware drive a quantum subsystem that runs algorithms aimed at solving problems whose cost grows super-polynomially with input size. Prototype processors from research groups and commercial vendors are now accessible through software platforms such as IBM Qiskit, which offers both a circuit simulator (Qiskit Aer) and cloud access to physical quantum devices. Designing such classical-quantum hybrid systems can draw on the equation-based modeling and Hardware-in-the-Loop (HIL) techniques implemented in tools like OpenModelica. This paper integrates IBM Qiskit into the Modelica ecosystem through the OpenModelica external C interface and the Qiskit C API. We exercise the resulting solver on a 0/1 knapsack test case drawn from patent portfolio analysis as an example of asset management.

Keywords: Quantum computing, knapsack problem, patent portfolio

1 Paradigm of Quantum Computing

Quantum computing departs from the von Neumann architecture that has defined classical digital computing for three-quarters of a century. Where a classical bit is fixed at 0 or 1, a quantum computer encodes information in a complex vector space and operates on it through superposition, entanglement, and interference. These ideas bear directly on computational complexity, cryptography, and high-performance computing (HPC).

1.1 Foundations and History

The conceptual foundations were laid in the early 1980s. Richard Feynman argued that simulating quantum systems on classical hardware incurs exponential overhead in the size of the state space, which motivated building computers whose substrate is itself quantum (Feynman 1982). Paul Benioff showed that a quantum-mechanical Turing machine was theoretically feasible (Benioff 1980), and David Deutsch described a universal quantum computer able to simulate any system that can be physically realized (Deutsch 1985).

These ideas stayed largely theoretical until 1994, when Peter Shor gave a polynomial-time quantum algorithm for integer factorization and discrete logarithms (Shor 1994). Since RSA and Diffie–Hellman rest on the hardness of factoring, Shor's result showed that a sufficiently large quantum computer could break widely deployed public-key cryptography. Two years later, Lov Grover gave a quantum search algorithm with a quadratic speedup for unstructured search (Grover 1996). Shor's and Grover's algorithms together set the field's expectation of useful quantum speedups.

1.2 Quantum Complexity Theory

Quantum computers do not solve hard problems by evaluating every answer in parallel, as is sometimes supposed. Instead, they evolve a quantum state so that the amplitudes of correct solutions reinforce through constructive interference while incorrect paths cancel.

The class *BQP* (Bounded-Error Quantum Polynomial-Time) captures the decision problems a quantum computer can solve in polynomial time with bounded error. Every problem in *P* lies in BQP, since classical logic can be expressed with reversible quantum gates such as the Toffoli (Nielsen and Chuang 2010). BQP overlaps *NP* but is believed distinct from it (Figure 1). Integer factorization, for example, lies in BQP and in *NP* yet, conjecturally, outside *P*. BQP is not, however, believed to contain the *NP*-complete problems. No pure quantum algorithm is known that solves combinatorial problems such as knapsack, the Traveling Salesman Problem, or SAT in polynomial time. For unstructured instances, Grover's algorithm gives only a quadratic speedup ($\mathcal{O}(2^{n/2})$ versus $\mathcal{O}(2^n)$), which shortens the runtime without changing the complexity class.

1.3 Hybrid HPC Ecosystem

Modern quantum hardware is integrated into traditional HPC environments rather than operated in isolation. Cloud-based Quantum-Computing-as-a-Service (QCaaS) models route bulk data processing and optimization loops to classical CPUs and GPUs, offloading specific non-classical subroutines to Quantum Processing Units (QPUs).

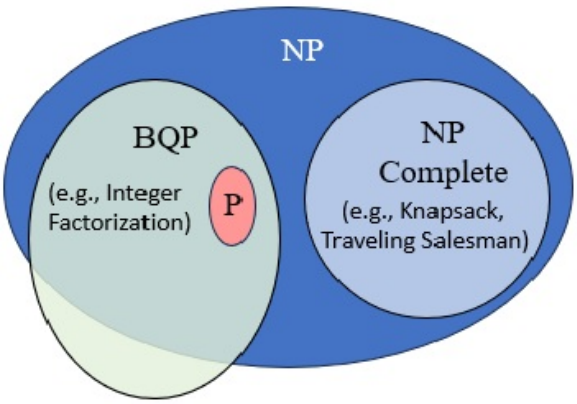


Figure 1. Containment relationships among complexity classes *P*, BQP, *NP*, and *NP*-complete. Adapted from (Sahu and Gupta 2023).

1.4 IBM Quantum Hardware and the Qiskit Software Stack

IBM provides cloud access to its fleet of superconducting quantum processors through Qiskit, an open-source Software Development Kit (SDK) organized as a multi-layered framework that abstracts the underlying hardware (Javadi-Abhari et al. 2024):

Circuit Construction (SDK): Qiskit allows developers to construct quantum circuits by initializing quantum registers and appending gates (Hadamard, CNOT, parameterized rotations).

Transpilation: Because each backend has its own qubit topology and native gate set, the Qiskit Transpiler rewrites abstract circuits to match these constraints, routing qubits via SWAPs (which exchange the states of two qubits) and lowering high-level gates into native instructions.

Execution Primitives: Qiskit exposes execution through two unified primitives. The *Sampler* returns the measurement-bitstring distribution of a circuit, suited to algorithms with discrete outputs (e.g., Grover-class search). The *Estimator* returns expectation values of user-supplied observables, used by variational algorithms such as the Variational Quantum Eigensolver (VQE).

Backends: A single Qiskit interface targets either the `qiskit-aer` simulator (statevector simulation with optional noise injection, limited by classical memory to roughly 30–40 qubits) or a physical IBM processor (e.g., Eagle®, Heron®, Osprey®) accessed over the cloud.

2 The Quantum Knapsack Problem

The Knapsack Problem is a central problem in combinatorial optimization and a clean model of a hard resource constraint: given a weight capacity and a set of items with values and weights, which subset maximizes total value without exceeding the capacity?

Its study, and that of its number-partition antecedents, spans more than a century (Mathews 1896), and the problem has served as a proving ground for the core techniques of combinatorial optimization — dynamic programming, branch-and-bound, relaxations, and approximation schemes (Kellerer et al. 2004) — growing from a logistical puzzle into a working tool for quantitative finance and, more recently, a target for quantum computing.

2.1 Classical Computational Complexity and Feasible Limits

To understand why the Knapsack Problem continues to challenge modern digital supercomputers despite decades of algorithmic optimization, one must examine its exact position within computational complexity theory.

2.1.1 Exponential Search Space

In its standard 0/1 formulation, an instance with n items presents a discrete choice for each item: either it is excluded (0) or included (1). This yields a total combinatorial search space of exactly 2^n possible subsets.

For a small portfolio of 20 assets, a classical computer evaluates $2^{20} \approx 1.05 \times 10^6$ combinations, a trivial task done in milliseconds. Scaling the pool to 100 assets raises the search space to $2^{100} \approx 1.27 \times 10^{30}$ combinations. Even at one trillion per second, an exhaustive search would take over 40 billion years, several times the age of the universe. That figure measures the exhaustive search space, not the practical difficulty of the standard instance: exact methods that sidestep enumeration (dynamic programming, branch and bound, and branch and cut) routinely dispatch 100-item 0/1 knapsack instances in a fraction of a second on commodity hardware (Kellerer et al. 2004).

2.1.2 Pseudo-Polynomial Time

Dynamic-programming algorithms such as Bellman's (Bellman 1957) solve the 0/1 Knapsack Problem in $\mathcal{O}(nW)$ time, where n is the number of items and W is the capacity. This looks like an efficient polynomial-time bound, but the appearance is misleading: the regime is called *pseudo-polynomial*.

The variable W is bounded not by the number of inputs but by the magnitude of a numeric value. In digital computers, a number W is represented in binary using $m = \log_2(W)$ bits. Expressed in terms of the actual length of the input data, the time complexity is therefore strictly exponential:

$$\mathcal{O}(n \cdot 2^m)$$

When capacity or asset values require high precision (currency to the cent, or fractional micro-shares), W grows very large, and classical solvers may run out of memory or time.

2.1.3 Correlated Instances and Reduction Failure

The practical "feasibility limit" of classical solvers depends on the structure of the data. In real asset allocation, values and their weights (costs or risks) are often strongly correlated: more valuable assets tend to cost more or carry higher risk.

Under strong correlation, standard reduction mechanisms such as bounding functions and the "core-problem" method (reduction to smaller, essential subproblems) break down. Pruning cannot eliminate sub-branches early, and solvers fall back to their worst-case exponential behavior. This is a hard ceiling for classical digital architecture.

2.1.4 Constrained Variants and Quantum Approaches

As classical improvements plateau against these limits, research has shifted toward more constrained variants that defeat standard reduction techniques. The Multi-Dimensional Knapsack Problem (MDKP) requires the chosen set to satisfy several capacity constraints at once, such as weight, volume, and budget. The Quadratic Knapsack Problem (QKP) assigns value not only to individual items but also to synergies between pairs of selected items.

To navigate these solution spaces, researchers have turned to quantum methods, several of which are under active investigation. Grover's search offers a provable quadratic speedup for unstructured search (Grover 1996), reducing the query cost of scanning high-dimensional combinations. The Quantum Approximate Optimization Algorithm (QAOA) is a hybrid method for Noisy Intermediate-Scale Quantum (NISQ) devices (Preskill 2018) that uses a classical outer loop to tune quantum parameters (Farhi et al. 2014). It can be adapted to knapsack constraints by "warm-starting" the circuit from the solution of a continuous relaxation, so the optimizer inherits the relaxation's approximation guarantees (Egger et al. 2021). Quantum annealing instead seeks the lowest-energy state of an Ising (equivalently, Quadratic Unconstrained Binary Optimization, or QUBO) formulation of the problem (Kadowaki and Nishimori 1998; Lucas 2014), the approach used by D-Wave hardware for large portfolio-balancing problems (Venturelli and Kondratyev 2019). Most recently, the Quantum Tree Generator prepares a superposition over a knapsack instance's feasible set that amplitude amplification then drives toward optimal selections (Wilkening, Lefterovici, Binkowski, Perk, et al. 2025), a method since extended to the quadratic and multidimensional variants (Wilkening, Lefterovici, Binkowski, Funck, et al. 2025).

Achieving a true *quantum advantage*, where a quantum computer cleanly outperforms the best classical supercomputers on a practical task, is not yet within reach here. Unlike integer factorization, which Shor's algorithm solves exponentially faster, the Knapsack Problem lacks a clean periodic structure, and current evidence suggests any quantum speedup is likely to be polynomial rather than exponential.

Consequently, near-term applications are expected to rely on hybrid architectures. Rather than replacing classical frameworks, Quantum Processing Units are likely to serve as coprocessors, invoked by classical CPUs to handle the hardest-to-optimize problem instances.

2.1.5 Knapsack Applications in Asset Management

The allocation of a finite capital budget among competing projects is the classic capital-rationing problem (Lorie and Savage 1955), and the knapsack model captures it directly: the "knapsack" is the budget or risk allowance, and the "items" are the candidate investments selected to maximize returns.

While traditional frameworks like Modern Portfolio Theory (MPT) focus heavily on statistical variance and covariance to build a continuous frontier, they often struggle with real-world, discrete operational limits. The knapsack model fills this gap by offering a structural, rule-based approach to explicit integer constraints, cardinality limits, and transaction friction.

This framework appears across corporate finance and wealth management. In capital budgeting, a firm with a fixed budget for new projects faces a binary fund-or-not decision for each one, and modeling it as a knapsack identifies the set of independent projects that maximizes aggregate Net Present Value (NPV) without breaching the ceiling. In asset-backed securitization, institutions pool illiquid assets such as mortgages, auto loans, and lease contracts into marketable securities whose profitability depends on which assets are chosen, a selection naturally posed as an MDKP with constraints on pool size, risk tranches, and yield. Portfolios of patents, trademarks, and other intangibles, increasingly central to corporate valuation, fit the knapsack formulation because acquisition and divestment decisions turn on time-varying cost and appreciation estimates.

2.1.6 Portfolio Binary and Cardinality Constraints

Selecting a basket of assets is rarely as simple as setting abstract percentage weights. Fund managers face strict operational limits. Minimum investment thresholds rule out holding an arbitrarily small fraction of every asset, and the 0/1 knapsack model captures this by forcing a binary include-or-exclude decision for each position. A cardinality cap, such as "at most 30 holdings," adds a linear constraint on the number of positions and contains transaction costs and oversight burden, a device studied widely in portfolio optimization (Chang et al. 2000). Redefining "weight" as an asset's volatility or Value-at-Risk and "value" as its expected return turns the knapsack into a tool for building maximum-yield portfolios under a hard risk budget. When assets are divisible into fractional shares, the problem relaxes to a continuous fractional knapsack, which admits a fast greedy solution and real-time capital weighting within a liquidity envelope.

Real markets demand optimization under heteroge-

neous constraints such as compliance mandates, transaction costs, and liquidity shifts. Whether solved by classical metaheuristics (such as genetic algorithms or reinforcement learning) or by hybrid quantum methods, the knapsack formulation remains a practical tool for disciplined capital allocation under hard constraints.

3 Patents as Intangible Assets

Managing patent collections as investable portfolios reframes intellectual property (IP) from a purely defensive legal instrument into a revenue-generating asset class. Applying portfolio-theory principles to IP lets executives and institutional investors diversify technology risk, build recurring licensing revenue, and surface latent balance-sheet value. This is a specific instance of a broader question: how the advent of fault-tolerant quantum computing can be expected to affect law and policy. For a survey of that landscape, see Myers (2026).

3.1 The Patent Asset Knapsack Problem

The 0/1 Knapsack Problem models a universal constraint: how to maximize the value of a choice when resources are strictly limited.

Consider a portfolio manager acquiring patents as intangible assets for a company that wants to make, use, and sell quantum processors. From a pool $P = \{p_1, p_2, \ldots, p_n\}$ of candidate patent families, she may acquire families up to a fixed cost budget W. Each item i has a value v_i (its utility) and a cost c_i (the resource cost of acquiring it). The objective is to select a subset whose total cost stays within the budget while the accumulated value of the selected families is as large as possible.

The "0/1" prefix specifies a strict binary constraint on item selection. For each item, the portfolio manager faces a binary choice:

0 (Exclude): Leave the item behind completely.

1 (Include): Acquire the entire legal rights of the patent.

Under this formulation, the legal rights conferred by a patent are treated as indivisible: a portfolio manager may not acquire a fractional share of a patent (e.g., rights limited to a single claim), nor may the same patent be selected more than once.

To express the 0/1 Knapsack Problem formally, we model the choices as a decision vector and set up an optimization framework consisting of an objective function and a capacity constraint:

Given:

- A set of items indexed from 1 to n.
- A vector of item values: $\mathbf{v} = [v_1, v_2, \ldots, v_n]$, where $v_i > 0$.
- A vector of item costs: $\mathbf{c} = [c_1, c_2, \ldots, c_n]$, where $c_i > 0$.
- A maximum cost budget: W.

We define a set of binary decision variables, x_i, for each item i:

$$x_i \in \{0,1\} \quad \forall i \in \{1,\ldots,n\}$$

The objective is to maximize the cumulative value of the chosen items:

$$\max \sum_{i=1}^{n} v_i x_i \tag{1}$$

The total cost of the selected patents must not exceed the acquisition budget. This is enforced by requiring the sum of the selected costs to be no greater than W:

$$\text{subject to } \sum_{i=1}^{n} c_i x_i \leq W \tag{2}$$

The computational complexity of the 0/1 Knapsack Problem is classified as *NP*-hard: there is no known classical algorithm that can find the exact optimal solution in polynomial time ($\mathcal{O}(n^k)$) for all possible instances. Because each of the n items has two choices (0 or 1), a brute-force search must evaluate all 2^n possible subsets, which becomes intractable beyond modest problem sizes.

4 OpenModelica & Qiskit Integration

This work pursues two objectives: to integrate the Qiskit C API as a callable computational engine within OpenModelica, exercised here in simulation (a self-contained statevector simulator, supplemented by an Aer noise study), with the exported OpenQASM 3 circuit description keeping a path open to physical IBM quantum hardware; and to use that integrated environment to demonstrate that an *NP*-hard combinatorial problem can be posed and solved within OpenModelica augmented by a quantum solver. We emphasize that this paper is a proof of concept: it demonstrates the feasibility of integrating Qiskit into the OpenModelica analysis environment. It is *not* an exercise in knapsack-algorithm development, and it makes no claim of quantum advantage, algorithmic or hardware-demonstrated. Those are separate endeavors to be pursued once a properly integrated OpenModelica-Qiskit platform has been established.

We demonstrate integration using a simple patent-portfolio Knapsack Problem as formulated in Section 3.1.

4.1 Quantum Brute-Force Enumeration

Although classical dynamic-programming solutions to the 0/1 Knapsack Problem exist (Section 2), the problem provides a natural vehicle for demonstrating quantum brute-force enumeration. An N-qubit Hadamard layer creates a uniform superposition over *all* 2^N candidate subsets simultaneously, after which classical post-processing on the statevector probabilities identifies the optimal feasible state. Recasting (1)–(2) in this enumeration form, we select a subset $S \subseteq \{0,\ldots,N-1\}$ that maximizes $\sum_{i\in S} v_i$ subject to $\sum_{i\in S} c_i \leq W$ for a budget W.

4.2 Prototype Implementation

MODKNAPSACK realizes this approach inside OpenModelica, a widely used open-source equation-based modeling tool (Fritzson 2015), by constructing and simulating a quantum brute-force circuit through the Qiskit C API and exposing the solver to OpenModelica through its external C function interface. A C solver (`knapsack.c`) drives the Qiskit C API (`libqiskit.so`) to construct the circuit, then runs a self-contained pure-C statevector simulator to evaluate it. The design spans three technical layers (Modelica, C, and the Qiskit library API) and contributes a correct, portable pattern for passing multiple typed outputs through OpenModelica's external C interface. Source code, together with compilation and linking instructions and a companion document detailing the implemented algorithms and circuits, is available at `https://github.com/Legal-Ingenuity/2026ModelicaConference`.

5 System Architecture

5.1 Layered Overview

Figure 2 shows the three-tier architecture. The *Modelica layer* consists of two entities in the package `ModKnapsack`: the external-function wrapper `ModCallKnapsack` and the simulatable model `KnapsackRun`. The *C layer* contains `callKnapsack` (the OpenModelica entry point) and `solve_knapsack` (the quantum solver proper). The *library layer* is the prebuilt Qiskit shared library `libqiskit.so`, accessed through the Qiskit C API headers.

5.2 Data Flow

At simulation time OpenModelica calls `ModCallKnapsack` with five inputs: the number of items `Npatents` (N), the value and cost arrays `Values`[N] and `Costs`[N], the budget `MaxCost` (W), and the circuit-drawing detail level `QDrawDetail`. On return, two outputs are consumed: the binary inclusion vector `PatentIndices`[N] and the execution status `ExStatus` (0 on success, -1 when no feasible subset exists, -2 on invalid input, and -3 when the instance exceeds the quantum solver's capacity, $N > N_{\max}$). A third output, `ActualCost`, is computed entirely within Modelica by summing `Costs`[i] for every i where `PatentIndices`[i] = 1.

6 The Quantum Circuit

6.1 Circuit Design Rationale

The circuit is an N-qubit *quantum brute-force enumerator*. Its design is deliberately simple: because the post-processor evaluates *every* 2^N basis state exhaustively, there is no advantage to biasing the initial state toward promising subsets. The circuit therefore places every qubit in an equal superposition, making all 2^N computational basis states equiprobable. This is the standard uniform-superposition preparation that also seeds more sophisticated algorithms such as Grover's search (Grover 1996).

For a reader new to quantum circuits, it helps to separate what the circuit does from what the surrounding algorithm does. The circuit's *sole* computational act is the Hadamard layer, whose purpose is to place the N item qubits into a uniform superposition that represents all 2^N candidate subsets at once—item i belongs to the subset encoded by basis state $|x\rangle$ exactly when bit i of x is 1. The barrier that follows is only a drawing separator and performs no operation, and the final measurement returns a *single*, randomly drawn subset into the `sel` register—not the optimum.

Crucially, the optimum is chosen by the *classical* post-processor rather than by the measurement: the pure-C statevector simulator exposes the amplitudes of all 2^N basis states, and an exhaustive scan keeps the highest-value subset whose cost stays within the budget (Section 8). The quantum register therefore *enumerates* the search space while the classical layer *selects* the answer, so this brute-force construction makes no claim of quantum speedup. Because it uses only single-qubit Hadamard gates and no entangling operations, the circuit exercises superposition but not entanglement. Entanglement enters only with the Grover solver whose results are reported in Section 11. A citation-backed walkthrough with flowcharts, written for readers unfamiliar with quantum-circuit notation, accompanies the source in the repository's algorithm companion.

6.2 Circuit Structure

The circuit uses two named registers:

- **item**$[0..N-1]$: quantum register, N qubits, one per item.
- **sel**$[0..N-1]$: classical register, N bits, measurement targets.

Three instruction layers follow in sequence:

1. **Superposition layer.** A Hadamard gate H is applied to every item qubit, transforming the initial state:

$$|0\rangle^{\otimes N} \xrightarrow{H^{\otimes N}} \frac{1}{\sqrt{2^N}} \sum_{x=0}^{2^N-1} |x\rangle. \tag{3}$$

   Basis state $|x\rangle$ encodes the subset in which item i is included iff bit i of x is 1.

2. **Barrier.** A barrier instruction spanning all N qubits acts as a visual separator between the superposition and measurement layers in circuit diagrams. It has no effect on the unitary evolution.

3. **Measurement layer.** Each qubit i is measured into classical bit i. After the H layer, every basis state is measured with equal probability $1/2^N$. The Born-rule outcome (Nielsen and Chuang 2010) is recorded in register **sel**: each basis state is observed with probability equal to its squared amplitude, $P(x) = |\alpha_x|^2$ (Eq. 4).

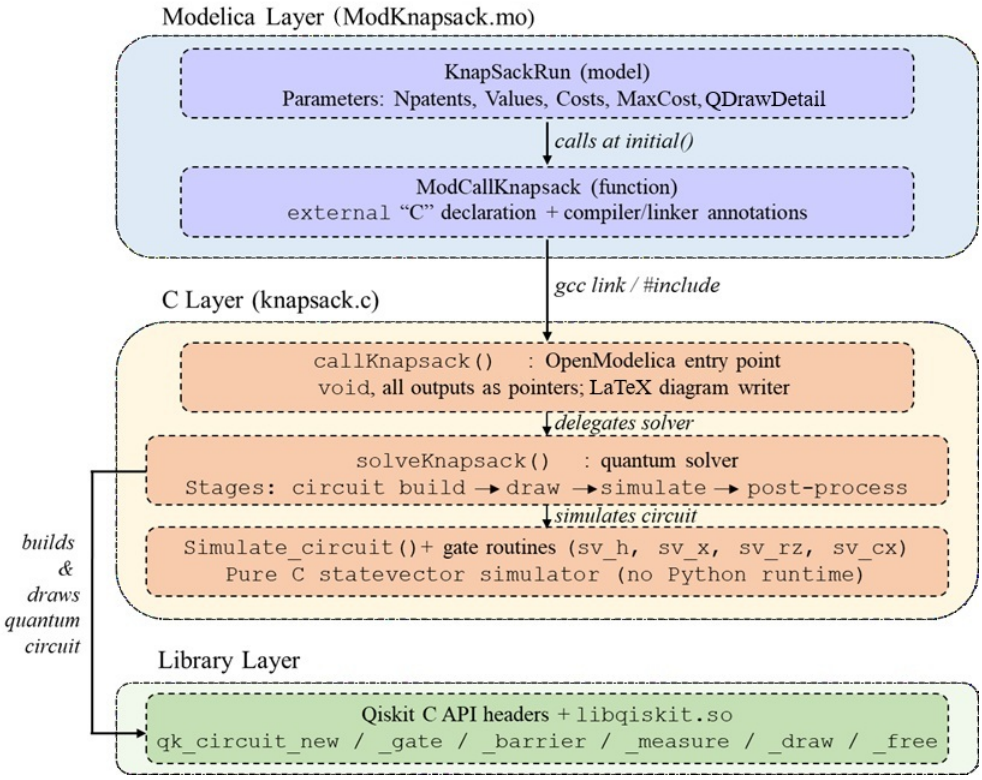


Figure 2. Three-tier architecture of MODKNAPSACK. Control flows top-down, and data flows back up via C pointer arguments.

6.3 Circuit Diagram

Figure 3 shows the circuit for a generic N-item problem, drawn directly in Ti*k*Z for portability. The `write_latex_circuit()` routine in `knapsack.c` emits an equivalent `quantikz` diagram to `printCircuitFile.txt`, one of several outputs selectable through the integer `QDrawDetail` detail level (ranging from no output, through text and schematic renderings, to a gate-level OpenQASM 3 export).

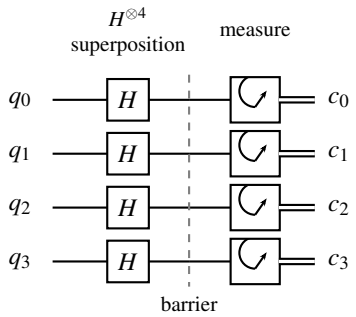


Figure 3. The N=4 quantum brute-force knapsack circuit. A Hadamard layer $H^{\otimes 4}$ places the register in a uniform superposition over all $2^4 = 16$ item-selection subsets (Eq. 3). A barrier (dashed line) separates the superposition layer from the measurement layer. Each qubit $q_i \equiv \texttt{item}[i]$ is measured into classical bit $c_i \equiv \texttt{sel}[i]$. For general N, the circuit extends to N qubit wires with a correspondingly wider barrier.

Table 1 summarizes the circuit parameters.

Table 1. Quantum circuit parameters.

Quantity	Value	Notes
Qubits	N	one per item
Classical bits	N	one per item
Hadamard gates	N	$H^{\otimes N}$ layer
Barriers	1	visual separator
Measurements	N	qubit $i \rightarrow$ bit i
Basis states	2^N	all item subsets
Statevector size	$2^N \times 16\,\text{B}$	complex128

7 The Qiskit C API

7.1 API Overview

The Qiskit C API is experimental code made available by IBM under an open-source license bundled in the Qiskit SDK source code.[1] Table 2 lists the API calls used in the prototype `knapsack.c` (Section 8).

7.2 Circuit Construction

Circuit construction in `solve_knapsack()` follows the ownership model of the Qiskit C API, in which each piece of data is "owned" by one specific "owner".

[1] To use the C API, the user must compile the Qiskit SDK source code using Rust, then build the C API separately using a makefile. See `https://quantum.cloud.ibm.com/docs/en/api/qiskit-c`. The underlying implementation is written in Rust, and the C API is a thin bridge over the Rust core.

Table 2. Qiskit C API functions used in the prototype `knapsack.c`.

Function	Purpose
`qk_circuit_new`	Allocate circuit
`qk_quantum_register_new`	Create q-register
`qk_classical_register_new`	Create c-register
`qk_circuit_add_quantum_register`	Attach q-register
`qk_circuit_add_classical_register`	Attach c-register
`qk_{quantum,classical}_register_free`	Free registers
`qk_circuit_gate`	Append gate
`qk_circuit_barrier`	Append barrier
`qk_circuit_measure`	Append measurement
`qk_circuit_draw`	Text-art drawing
`qk_circuit_num_{qubits,clbits}`	Metadata query
`qk_circuit_num_instructions`	Instruction count
`qk_circuit_instruction_kind`	Instruction type
`qk_circuit_get_instruction`	Read instruction
`qk_circuit_instruction_clear`	Free instruction
`qk_circuit_free`	Deallocate circuit
`qk_param_str / qk_str_free`	Param serialization

The construction shown here is the prototype enumerator's, preserved in the repository alongside the successor solver whose results Section 11 reports. The successor builds its deeper Grover circuit through the same API, sized at creation rather than assembled from register objects. An empty `QkCircuit` is created with `qk_circuit_new(0,0)`, after which named registers are constructed, attached, and immediately released (the circuit retains its own copy):

Listing 1. Qiskit C API circuit construction (abridged).

```
QkCircuit *qc = qk_circuit_new(0, 0);
QkQuantumRegister  *qreg =
    qk_quantum_register_new((uint32_t)N,
        "item");
QkClassicalRegister *creg =
    qk_classical_register_new((uint32_t)N
        , "sel");
qk_circuit_add_quantum_register(qc, qreg)
    ;
qk_circuit_add_classical_register(qc,
    creg);
qk_quantum_register_free(qreg);
qk_classical_register_free(creg);
for (int i = 0; i < N; i++) {
    uint32_t q[1] = {(uint32_t)i};
    qk_circuit_gate(qc, QkGate_H, q, NULL
        );
}
uint32_t all_q[MAX_ITEMS];
for (int i = 0; i < N; i++) all_q[i] = (
    uint32_t)i;
qk_circuit_barrier(qc, all_q, (uint32_t)N
    );
for (int i = 0; i < N; i++)
    qk_circuit_measure(qc, (uint32_t)i, (
        uint32_t)i);
```

8 `knapsack.c` Implementation

The code in this section documents the original brute-force enumerator that is the subject of this paper. The GitHub repository's current `knapsack.c` has since been developed into the realized Grover solver and multi-format circuit exporter (the `QDrawDetail` drawing levels and OpenQASM 3 export). The solver's measured results are reported in Section 11. It therefore differs in structure and in its `solve_knapsack` internals from the listings here, which describe the earlier prototype.

8.1 Module Structure

`knapsack.c` is organized into five sections:

1. **Pure-C statevector simulator** (`sv_alloc`, `sv_h`, `sv_x`, `sv_y`, `sv_rz`, `sv_cx`).
2. **Circuit walker** (`simulate_circuit`).
3. **Quantum knapsack solver** (`solve_knapsack`).
4. **Module to produce a circuit diagram in LaTeX** (`write_latex_circuit`).
5. **OpenModelica entry point** (`callKnapsack`).

All internal functions are declared `static`, confining their linkage to the translation unit. Only `callKnapsack` carries external linkage and is visible to the OpenModelica-generated simulation binary.

8.2 Pure-C Statevector Simulator

Sections 1 and 2 implement a self-contained statevector simulator that requires no Python runtime or simulation back-end. A heap-allocated array of 2^N `double complex` values represents the amplitudes α_x, $x \in \{0,\ldots,2^N-1\}$, initialized to the all-zero state $|0\rangle^{\otimes N}$. Bit i of index x corresponds to qubit i (qubit 0 = LSB).

Each single-qubit gate routine iterates over 2^{N-1} amplitude pairs (i, j) where $j = i\,|\,(1 \ll t)$ for target qubit t, applying the gate matrix to both amplitudes in place. The supported gate set ($H, X, Y, Z, S, T, R_Z(\theta)$, Phase, CX) is more general than the brute-force circuit requires—which emits only the Hadamard layer—keeping the simulator reusable for richer circuits.

`simulate_circuit()` retrieves each instruction from the `QkCircuit` object via `qk_circuit_get_instruction()`, dispatching to the appropriate gate routine. Barrier, measurement, reset, and delay instructions are detected by `qk_circuit_instruction_kind()` and silently skipped. Probabilities are derived from the final amplitudes using the Born rule:

$$P(x) = |\alpha_x|^2 = \mathrm{Re}(\alpha_x)^2 + \mathrm{Im}(\alpha_x)^2. \tag{4}$$

After the $H^{\otimes N}$ layer all 2^N amplitudes equal $1/\sqrt{2^N}$, yielding $P(x) = 1/2^N$ uniformly.

8.3 Quantum Knapsack Post-Processor

With $P(x) = 1/2^N$ for all x, the measurement is a uniform random draw over all subsets. Rather than sampling a single outcome, the prototype's `solve_knapsack()` performs a deterministic classical *exhaustive search* over all 2^N basis states:

$$x^* = \arg\max_x \sum_{i=0}^{N-1} v_i \cdot b_i(x) \quad \text{s.t.} \quad \sum_{i=0}^{N-1} c_i \cdot b_i(x) \le W, \quad (5)$$

where $b_i(x) = (x \gg i)\,\&\,1 \in \{0,1\}$ is the i-th bit of x. The optimal basis state x^* is decoded into a binary indicator vector such that bit i of x^* yields `PatentIndices`$[i] \in \{0,1\}$. In the GitHub repository's current `knapsack.c`, this exhaustive search is retained only as an independent ground-truth check. The subset selection itself is performed by the realized Grover / Dürr–Høyer search (Dürr and Høyer 1996), whose results are reported in Section 11.

Single-inclusion guarantee. The bitmask encoding assigns each item to a unique bit position. Because a bit can be either 0 or 1 but is not set twice, no item can appear more than once in the selected subset. This structural property is made explicit by a `seen[]` guard in `solve_knapsack()`, which records each decoded index and permits at most one inclusion per item.

The total cost of the selected subset is computed in Modelica (see Section 9) to avoid the calling-convention issue that arises when C return values and scalar pointer outputs are mixed in OpenModelica's external C interface.

9 The OpenModelica C Interface

9.1 Package and Function Structure

`ModKnapsack.mo` defines a Modelica `package` with two members: the external-function wrapper `ModCallKnapsack` and the simulatable model `KnapsackRun`.

Listing 2. KnapsackRun algorithm section (abridged).

```
when initial() then
  (PatentIndices, ExStatus) :=
    ModCallKnapsack(Npatents, Values, Costs,
                    MaxCost, QDrawDetail);
  ActualCost := 0.0;
  for i in 1:Npatents loop
    if PatentIndices[i] == 1 then
      ActualCost := ActualCost + Costs[i];
    end if;
  end for;
  // ... print inputs and outputs ...
end when;
```

Binding the call to the initialization event is deliberate: the quantum solver acts here as an initialization-time co-processor, the same pattern by which a deployed model would consult a QPU before or between transients, and the Hardware-in-the-Loop direction of Section 12 extends exactly this call into the running simulation.

9.2 External Function Declaration

`ModCallKnapsack` is declared as a Modelica `function` with an `external "C"` clause that provides the explicit argument list:

Listing 3. ModCallKnapsack external declaration.

```
function ModCallKnapsack
  input  Integer Npatents;
  input  Real    Values[Npatents];
  input  Real    Costs[Npatents];
  input  Real    MaxCost;
  input  Integer QDrawDetail;
  output Integer PatentIndices[Npatents];
  output Integer ExStatus;
  external "C"
    callKnapsack(Npatents, Values, Costs,
        MaxCost,
                 PatentIndices, ExStatus,
                     QDrawDetail)
  annotation( Include = ...,
              IncludeDirectory = ...,
              Library = ...,
              LibraryDirectory = ... );
end ModCallKnapsack;
```

10 Compiler & Linker Specifications

10.1 Annotation Mechanism

OpenModelica resolves build dependencies for external C functions through four standard Modelica annotations attached to the `external "C"` clause. Table 3 lists the annotations used in `ModCallKnapsack` and the corresponding `gcc` flags they generate.

Table 3. Compiler/linker annotations and the `gcc` flags they produce.

Annotation	Example value	Effect
`Include`	`#include ".../knapsack.c"`	Embeds source in compilation unit
`IncludeDirectory`	`.../qiskit/dist/c/include`	Adds `-I` flag for Qiskit headers
`Library`	`{"qiskit","m"}`	Adds `-lqiskit -lm`
`LibraryDirectory`	`.../qiskit/dist/c/lib`	Adds `-L` flag for `libqiskit.so`

10.2 Source-Inclusion Strategy

Using the `Include` annotation to embed a `#include` directive for a full `.c` source file, rather than a pre-built library, is an intentional architectural choice with the following advantages:

1. **No separate build step.** OpenModelica's own `gcc` invocation handles the entire compilation at simulation time, so no Makefile or CMake is required.
2. **Single translation unit.** All `static` functions in `knapsack.c` are compiled with the generated simulation code, allowing the compiler to inline gate routines aggressively and simplifying symbol resolution.
3. **No symbol conflicts.** Only `callKnapsack` carries external linkage, while all helpers use `static` linkage and do not pollute the global symbol namespace.

The principal trade-off is recompilation cost: each `simulate()` call re-compiles `knapsack.c`. For production use, compiling `knapsack.c`

into a shared library and switching to the `Library`/`LibraryDirectory` pattern would amortize this cost.

10.3 Runtime Library Discovery

The `LibraryDirectory` annotation generates a `-L` linker flag that resolves the location of `libqiskit.so` at *link time*. At *run time*, the Linux dynamic loader searches for `libqiskit.so` through the standard mechanism: `LD_LIBRARY_PATH`, `/etc/ld.so.cache`, and system default paths. Since `libqiskit.so` is installed in a non-standard project directory, the user must extend the search path before simulation:

Listing 4. Runtime library path configuration.

```
export LD_LIBRARY_PATH=\
  <project_path>/qiskit/dist/c/lib:
      $LD_LIBRARY_PATH
```

Alternatively, an `RPATH`/`RUNPATH` entry could be embedded in the executable at link time via a `-Wl,-rpath` linker flag, reducing reliance on `LD_LIBRARY_PATH`. (We use the `LD_LIBRARY_PATH` approach here.)

11 Preliminary Results

The integration ships with a default four-item test case ($\mathbf{v} = [3,4,5,7]$, $\mathbf{c} = [2,3,4,5]$, budget $W = 6$) that a new user can run to confirm a working installation. With `libqiskit.so` on the library path, simulating the model from the project root with `omc run_knapsack.mos` constructs and evaluates the quantum circuit and returns `PatentIndices` $= [1,0,1,0]$ (items 0 and 2 selected) and `ExStatus` $= 0$, with `ActualCost` $= 6.0$ computed in Modelica. The selected subset attains the optimal total value of 8.0 within the budget, matching the classical optimum and confirming that the OpenModelica $\rightarrow$ C $\rightarrow$ Qiskit toolchain is wired correctly end-to-end.

The brute-force Hadamard circuit scales as 2^N, which confines it to small instances (roughly 20–25 items before classical memory is exhausted). Building on this integration, we have also prototyped a structured replacement: a Grover-based amplitude-amplification solver that performs the subset selection *in-circuit* (Figure 4), pairing a reversible cost/value oracle with quantum maximum-finding (Dürr and Høyer 1996; Grover 1996; Nielsen and Chuang 2010). It reproduces the optimal selection across the validation test cases, with the classical exhaustive search kept only as an independent ground-truth check and no claim of quantum advantage. On the four-item test case the realized solver recovers the optimum with 59 oracle calls over a best-of-three schedule (measured, deterministic seed) against $2^4 = 16$ classical evaluations for the exhaustive scan — at this scale the quadratic envelope has, as expected, not yet paid for itself.

To probe robustness beyond the noiseless Aer baseline, we used the OpenQASM 3 export path to sample the structured circuit under an Aer depolarizing noise model: the probability of recovering the optimal subset degrades smoothly as the two-qubit gate error rate rises, and the noise-tolerance threshold scales *inversely* with the number of entangling gates in the oracle, so deeper instances demand proportionally lower error rates.

Reading Figure 4 from left to right, the oracle U_f encodes the solver's selection criterion as six reversible steps on the ancilla registers: (1) accumulate the subset's total cost $\sum_i c_i x_i$ into the `cost` register and (2) its total value $\sum_i v_i x_i$ into the `value` register; (3) test the cost against the budget ($\leq W$) and (4) the value against a running threshold ($\geq T$), setting the feasibility and value flags; (5) *phase-mark* the subsets that are simultaneously feasible and high-value; and (6) *uncompute* the arithmetic so every ancilla returns to $|0\rangle$ and only the phase mark remains. The diffuser D then amplifies the marked amplitudes. One oracle–diffuser pass is a single Grover iteration, repeated under the BBHT schedule (Boyer et al. 1998) and wrapped in Dürr–Høyer maximum-finding (Dürr and Høyer 1996) to ratchet the threshold T toward the optimum. The repository's algorithm companion steps through this pipeline with flowcharts.

These results bound the regime in which the present integration operates, consistent with the scope stated in Section 4. The demonstration establishes that the problem can be posed and solved through the integrated toolchain. Algorithmic scaling to realistic portfolio sizes is deliberately deferred.

12 Conclusion

12.1 Contributions

MODKNAPSACK demonstrates a complete vertical integration from an equation-based Modelica model, through OpenModelica's external C interface, to a C solver that constructs and inspects a quantum circuit via the Qiskit C API. To the author's knowledge, this is the first published integration of the Qiskit C API into a Modelica platform. The key architectural contributions are:

- A concrete, runnable demonstration of the OpenModelica `Include` / `IncludeDirectory` / `Library` / `LibraryDirectory` annotation quadruple applied to the Qiskit C API. Because that API is a C ABI (Application Binary Interface) bridge over a Rust implementation, the demonstration establishes a build-and-link pattern by which a Rust-backed quantum-software stack can be invoked from OpenModelica.
- A portable calling-convention pattern for OpenModelica external C functions with multiple typed outputs including the `void`-return / all-pointer pattern, with array and scalar outputs both passed by pointer, as demonstrated by `callKnapsack`.
- A self-contained pure-C statevector simulator that evaluates Qiskit circuit objects without requiring a

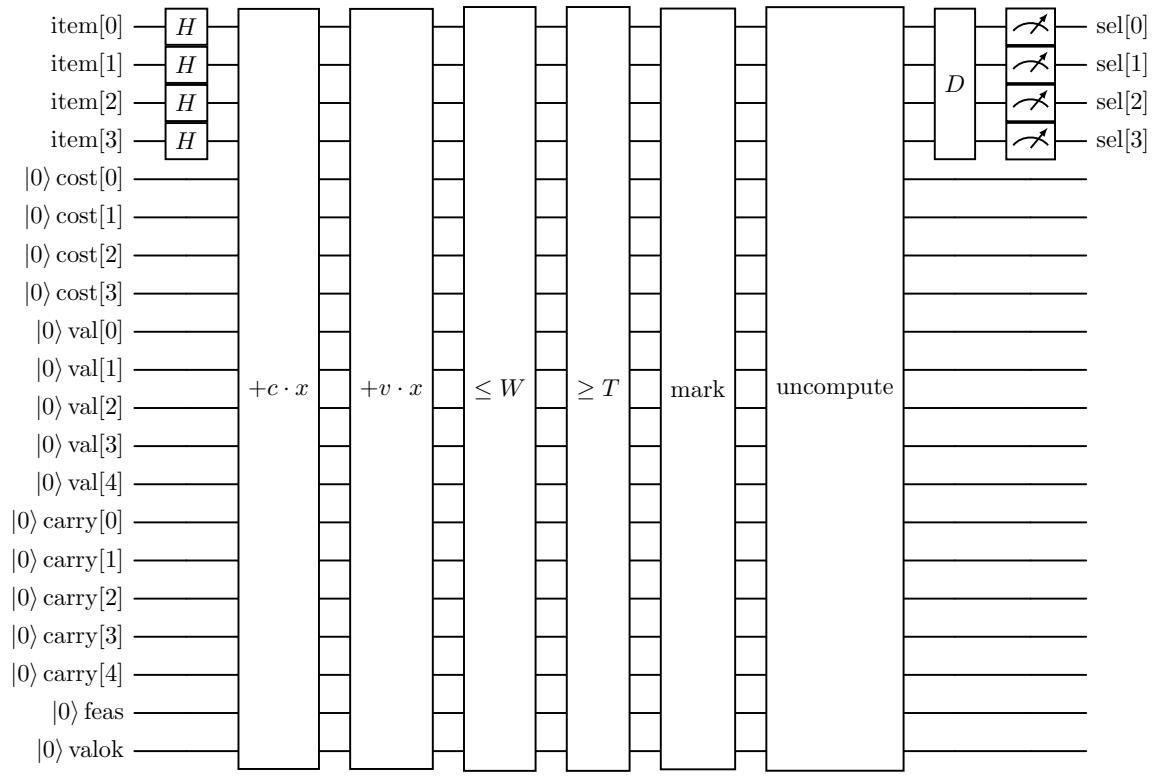


Figure 4. Register-level schematic of the realized Grover knapsack solver for the N=4 test case, emitted by the `QDrawDetail` circuit exporter in `knapsack.c`. The oracle–diffuser pipeline is walked through in the text. Only the `item` register is measured (into `sel`). The cost, value, and carry ancillae are $|0\rangle$-initialized and restored.

Python runtime, enabling the integration to run in any environment where `libqiskit.so` is available.

- A worked end-to-end test case, the patent-portfolio 0/1 Knapsack Problem, demonstrating that an *NP*-hard combinatorial optimization problem can be posed in Modelica syntax and solved through an embedded quantum-circuit subsystem — transparent enumeration with classical selection in the prototype presented here, and full in-circuit Grover selection in the repository's successor solver.

12.2 Future Work

Several directions remain open. Because the present study aimed to establish the viability of the integration between OpenModelica and Qiskit through their respective C-language APIs, it used only small 0/1 knapsack cases chosen for code validation. The most immediate priority is now to exercise the method on realistic patent portfolios from the quantum-computing space, drawing on intellectual-property sources such as the official USPTO repository.

The principal open challenge is scaling the structured solver of Section 11, or alternatives such as QAOA (Farhi et al. 2014), to industrially relevant portfolio sizes.

Execution on real IBM quantum hardware through the Qiskit C API has not yet been demonstrated. The solver keeps that path open by isolating circuit execution behind a single measurement "seam" and exporting any constructed circuit to OpenQASM 3, so the same portable description can be dispatched to the Aer simulator or, later, to a physical IBM QPU backend (e.g., Heron®) through `qiskit-ibm-runtime`, with shot-based sampling replacing the statevector post-processor. Validating that interface on hardware comes next.

On the modeling side, the patent-portfolio formulation currently treats patent values as static scalars. Richer variants, such as time-varying valuations, citation-network synergies (the Quadratic Knapsack variant), or multi-dimensional budget constraints (MDKP), would bring it closer to the realities of commercial IP management.

The OpenModelica integration also invites Hardware-in-the-Loop (HIL) workflows, in which a quantum co-processor is invoked asynchronously during a mechatronic simulation, opening possibilities for quantum-accelerated real-time optimization in cyberphysical systems. Because the correct operation of a hardware quantum circuit can depend sensitively on QPU temperature, such workflows even admit system-level control design in which the cryogenic subsystem itself becomes the hardware in the loop.

The `Include`-based single-translation-unit strategy is

convenient for prototyping but recompiles `knapsack.c` at every `simulate()` call. Migrating it to a pre-built shared library and benchmarking the compilation overhead would quantify that cost and ease the path to production deployment.

Incorporating device-calibrated noise models beyond the depolarizing study of Section 11, and folding the resulting stochasticity into the HIL analyses noted above, is a further important direction.

Acknowledgment

The author would like to thank the law firm, *McCormick, Paulding & Huber PLLC*, for their support.

During the preparation of this work the author used Anthropic's Claude (LLM chosen appropriate to task and cost, via Claude Code) as a development and editing assistant: to help resolve build and integration issues with the experimental Qiskit C API, to generate and debug test-harness scripts, and to assist with drafting and editing portions of the manuscript and its LaTeX representation. All algorithmic design, code, mathematical formulations, and text were directed, independently reviewed, and verified by the author, who retains sole intellectual property ownership of, and full responsibility for, the final work.

References

Bellman, Richard (1957). *Dynamic Programming*. Princeton, NJ: Princeton University Press.

Benioff, Paul (1980). "The Computer as a Physical System: A Microscopic Quantum Mechanical Hamiltonian Model of Computers as Represented by Turing Machines". In: *Journal of Statistical Physics* 22.5, pp. 563–591. DOI: 10.1007/BF01011339.

Boyer, Michel, Gilles Brassard, Peter Høyer, and Alain Tapp (1998). "Tight Bounds on Quantum Searching". In: *Fortschritte der Physik* 46.4–5, pp. 493–506.

Chang, T.-J., N. Meade, J. E. Beasley, and Y. M. Sharaiha (2000). "Heuristics for Cardinality Constrained Portfolio Optimisation". In: *Computers & Operations Research* 27.13, pp. 1271–1302. DOI: 10.1016/S0305-0548(99)00074-X.

Deutsch, David (1985). "Quantum Theory, the Church-Turing Principle and the Universal Quantum Computer". In: *Proceedings of the Royal Society of London. Series A, Mathematical and Physical Sciences* 400.1818, pp. 97–117. DOI: 10.1098/rspa.1985.0070.

Dürr, Christoph and Peter Høyer (1996). "A Quantum Algorithm for Finding the Minimum". In: *arXiv preprint arXiv:quant-ph/9607014*. DOI: 10.48550/arXiv.quant-ph/9607014.

Egger, Daniel J., Jakub Mareček, and Stefan Woerner (2021). "Warm-Starting Quantum Optimization". In: *Quantum* 5, p. 479. DOI: 10.22331/q-2021-06-17-479.

Farhi, Edward, Jeffrey Goldstone, and Sam Gutmann (2014). "A Quantum Approximate Optimization Algorithm". In: *arXiv preprint arXiv:1411.4028*. DOI: 10.48550/arXiv.1411.4028.

Feynman, Richard P. (1982). "Simulating Physics with Computers". In: *International Journal of Theoretical Physics* 21.6–7, pp. 467–488. DOI: 10.1007/BF02650179.

Fritzson, Peter (2015). *Principles of Object-Oriented Modeling and Simulation with Modelica 3.3: A Cyber-Physical Approach*. 2nd. Wiley-IEEE Press. ISBN: 978-1-118-85912-4.

Grover, Lov K. (1996). "A Fast Quantum Mechanical Algorithm for Database Search". In: *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, pp. 212–219. DOI: 10.1145/237814.237866.

Javadi-Abhari, Ali et al. (2024). "Quantum Computing with Qiskit". In: *arXiv preprint arXiv:2405.08810*. DOI: 10.48550/arXiv.2405.08810.

Kadowaki, Tadashi and Hidetoshi Nishimori (1998). "Quantum Annealing in the Transverse Ising Model". In: *Physical Review E* 58.5, pp. 5355–5363. DOI: 10.1103/PhysRevE.58.5355.

Kellerer, Hans, Ulrich Pferschy, and David Pisinger (2004). *Knapsack Problems*. Berlin, Heidelberg: Springer. DOI: 10.1007/978-3-540-24777-7.

Lorie, James H. and Leonard J. Savage (1955). "Three Problems in Rationing Capital". In: *The Journal of Business* 28.4, pp. 229–239. DOI: 10.1086/294081.

Lucas, Andrew (2014). "Ising Formulations of Many NP Problems". In: *Frontiers in Physics* 2, p. 5. DOI: 10.3389/fphy.2014.00005.

Mathews, G. B. (1896). "On the Partition of Numbers". In: *Proceedings of the London Mathematical Society* s1-28.1. Read 13 May 1897, pp. 486–490. DOI: 10.1112/plms/s1-28.1.486.

Myers, Mark R. (2026). *A Survey of Quantum Computing for the Working Attorney at the Dawn of the Fault-Tolerant Era*. Preprint. Zenodo. DOI: 10.5281/zenodo.22031924.

Nielsen, Michael A. and Isaac L. Chuang (2010). *Quantum Computation and Quantum Information*. 10th Anniversary. Cambridge: Cambridge University Press.

Preskill, John (2018). "Quantum Computing in the NISQ Era and Beyond". In: *Quantum* 2, p. 79. DOI: 10.22331/q-2018-08-06-79.

Sahu, Himanshu and Hari Prabhat Gupta (2023). "Quantum Computing Toolkit: From Nuts and Bolts to Sack of Tools". In: *arXiv preprint arXiv:2302.08884*. DOI: 10.48550/arXiv.2302.08884.

Shor, Peter W. (1994). "Algorithms for Quantum Computation: Discrete Logarithms and Factoring". In: *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE Computer Society, pp. 124–134. DOI: 10.1109/SFCS.1994.365700.

Venturelli, Davide and Alexei Kondratyev (2019). "Reverse Quantum Annealing Approach to Portfolio Optimization Problems". In: *Quantum Machine Intelligence* 1, pp. 17–30. DOI: 10.1007/s42484-019-00001-w.

Wilkening, Sören, Andreea-Iulia Lefterovici, Lennart Binkowski, Marlene Funck, et al. (2025). "A Quantum Search Method for Quadratic and Multidimensional Knapsack Problems". In: *arXiv preprint arXiv:2503.22325*. DOI: 10.48550/arXiv.2503.22325.

Wilkening, Sören, Andreea-Iulia Lefterovici, Lennart Binkowski, Michael Perk, Sándor P. Fekete, and Tobias J. Osborne (2025). "A Quantum Algorithm for Solving 0-1 Knapsack Problems". In: *npj Quantum Information* 11.1, p. 146. DOI: 10.1038/s41534-025-01097-8.